Image Synthesis of EV Travel Data with Generative Adversarial Networks

1st Matsuru Hoshi
School of Electrical Engineering and Computer Science
University of Ottawa
Ottawa, Canada
mhosh084@uottawa.ca

2nd Nan Chen
School of Electrical Engineering and Computer Science
University of Ottawa
Ottawa, Canada
nchen@uottawa.ca

***Abstract*—Electric vehicle (EV) travel data are often limited, and/or difficult to collect in large quantities. In this paper, we study the use of Generative Adversarial Networks (GAN), in particular FastGAN, to generate traffic map images of a small few-shot dataset on EV travel data. We propose a workflow capable of turning a dataset with frequently travelled locations into full-fledged city traffic maps. Two important things are done to achieve this: we simulate plausible routes between travel locations with a routing API, and we reparameterize the curve of routes so points more accurately represent traffic density. With 588 training samples we are capable of training the model under 1h and show its ability to expand dataset sizes for limited EV travel data.**

***Index Terms*—Generative Adversarial Networks (GANs), electric vehicle, data augmentation, traffic density, image synthesis.**

I. Introduction

Increased electric vehicle (EV) demand in recent years has encouraged the development of new and better EV charging infrastructure. While there are many determining factors concerning EV adoption by consumers, infrastructure remains a strong component [1], [2]. Since adequate charging infrastructure is an important determinant of successful EV adoption, it becomes necessary to develop efficient methods to predict where and how much infrastructure to build. Understanding travel trajectory and traffic patterns become thus important in deciding how to build charging infrastructure [3].

When it comes to predicting EV travel patterns, the biggest challenge arises from the limited public data on individual EV route trajectory of EV drivers within city maps. Experimental data collection is possible but often time-consuming, at times offering insufficient variability between collected travels. Generative Adversarial Networks (GANs) [4] have proven to be incredibly capable of synthesizing images, image editing, and generating art. In particular, in this paper we make use of FastGAN [5], a model shown to achieve impressive results on par with other well-performing models of its time [6]. Of interest to us, the key advantage of FastGAN is that it can be trained with small "few-shot" datasets and limited GPU to generate images of up to 1024×1024. In the FastGAN paper [5], the authors report training the model on 13 different datasets, featuring a wide variety of themes ranging from Pokémon to Obama. The results show FastGAN is very capable at generalizing across different datasets and suggests it could perform well on our EV travel dataset, despite no training on map data in the original paper. Another issue we encounter from the data is the limited number of points throughout the EVs' travels, only collected for frequently visited "landmarks". This leaves us to reconstruct a plausible route in order to build driving trajectories for the EVs and from that create a dataset.

In this paper, our goal is to build a workflow to expand datasets created from limited geographical datapoints by using a quickly converging model such as FastGAN. As shown in Fig. 1, this first requires a small dataset of trip coordinates for common destinations of EVs. These data are transformed into satellite-like map image data with visual route density information. Given the small size of the dataset created, itself directly dependent on the size of the collected data, we use it to train the FastGAN model to expand more varied generalized samples of the original dataset. We summarize our contributions as follows:

- We create a dataset with simulations of plausible routes between the limited "landmark" coordinate datapoints using the OpenRouteService API. The open-source service is run on a local Docker container, so as to avoid the web API rate limit and effectively perform the task.
- We adjust the simulated coordinate datapoints by interpolating the route with a B-spline and reparameterizing the

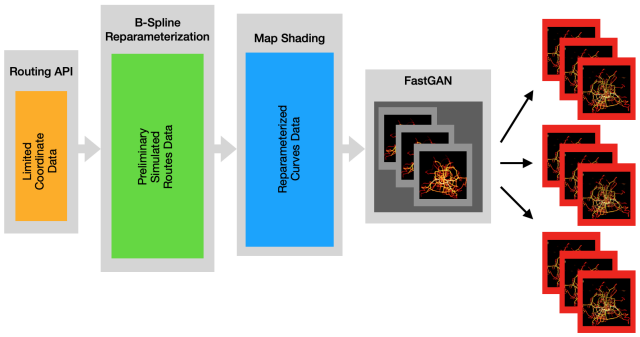


Fig. 1: The workflow to generate extensive map image data from limited coordinate data.

curve of the route such that the datapoints are somewhat equidistant from each other along the curve. This ensures the map image has only commonly used routes brighter than less common routes. From our adjusted coordinate data, we then create a proper image dataset of the map data.

- We train FastGAN on our own shaded map dataset and show the model's generalization abilities in creating large and varied samples of our map data.

II. Literatue Review

A. A Primer on GANs

GANs have been a well-researched architecture for image synthesis in recent years. First introduced in [4], they make use of two neural networks, a generator and a discriminator model, corresponding to a minimax two-player game. This is represented in the value function (1). The discriminator, nothing more than a binary classifier, has the goal of maximizing the probability of assigning the correct label to an image. The generator on the other hand, a convolutional neural network (CNN), must try to fool the discriminator into thinking the image it has generated is real.

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}(x)}[\log D(x)] + \mathbb{E}_{z\sim p_z(z)}[\log(1-D(G(z)))] \quad (1)$$

Introduced alongside the original paper [4], the value function represents the minimax game between the discriminator and the generator. We now explain its components. We note that x is a sample from the original data, of distribution p_{data} and z from input noise p_z. Both $\mathbb{E}$ terms represent the expected value over their respective distributions. $D(x)$ is given as the probability that x came from the real data rather than the generated [4]. It thus ranges between 0 and 1. This is in contrast with $G(z)$, which is instead a function that returns us with a generated sample (D and G are in fact our neural networks, but we may interpret them as functions returning some result).

We now explain what the minimax game consists of by analyzing the cases for when the value function is minimized or maximized. We recall that D is a probability distribution taking on values in the range $[0,1]$. Per the definition, real samples should be classified as real, meaning $D(x)$ must tend to 1, making the first term of the value function evaluate to 0 (where $\log 1 = 0$) . As for $G(z)$, recall that it returns us generated samples. It follows that $D(G(z))$ must tend to 0, resulting in the second term of the function to evaluate to 0. Note that any value lower than 1 for $D(x)$ and greater than 0 for $D(G(z))$ will yield a lower value for the value function, thus maximizing at 0. We now turn to the minimization of the value function. The first term does not depend on G and may be disregarded. Since the goal of the generator is to pass generated samples as real (and "fool" the discriminator) it succeeds in doing so when $D(G(z))$ tends to 1. In this case, the second term of the function tends to $-\infty$ and is thus minimized, as required. As such, D is trained to maximize the probability of correctly labelling the real and generated samples, and G is trained to "fool" D. Theoretically, the two neural networks compete until they reach a global optima where $p_g = p_{data}$, as shown in the original paper. This concludes the analysis of the GAN definition given in [4].

We now turn to a formal explanation that adjusts to the deep learning landscape. In this scenario, we look at three sets of inputs (we employ the same notation as [8] to facilitate readability). We have our set of real training data $\{\mathbf{x}_i\}$ of size I where a single sample is $\mathbf{x}_i$, our generated set $\{\mathbf{x}_j^*\}$ of size J where a sample is $\mathbf{x}_j^*$, and finally our latent vector $\mathbf{z}_j$ from which a sample $\mathbf{x}_j^*$ is generated. As stated earlier, the generator and discriminator comprise of a network each. The generator is defined as a network where $\mathbf{x}_j^* = \mathbf{g}[\mathbf{z}_j, \boldsymbol{\theta}]$ with a trainable parameter $\boldsymbol{\theta}$. We add the discriminator's network $\mathbf{f}[\bullet, \boldsymbol{\phi}]$, with its trainable parameter $\boldsymbol{\phi}$, its goal to determine whether a sample is real or generated, as we have seen. In this new context, two loss functions based on 1 are defined, one for the discriminator and one for the generator. Per [8] we have:

$$L[\boldsymbol{\phi}] = \sum_j -\log[1-\text{sig}[\mathbf{f}[g[\mathbf{z}_j,\boldsymbol{\theta}],\boldsymbol{\phi}]]] - \sum_i \log[\text{sig}[\mathbf{f}[\mathbf{x}_i,\boldsymbol{\phi}]]] \quad (2)$$

$$L[\boldsymbol{\theta}] = \sum_j \log[1-\text{sig}[\mathbf{f}[\mathbf{g}[\mathbf{z}_j,\boldsymbol{\theta}],\boldsymbol{\phi}]]] \quad (3)$$

[1]

Note how the loss functions are defined as functions of the trainable parameters of each respective network. The author in [8] formulates the loss function slightly differently from the value function in [4], however, the same reasoning holds. The main difference is that with the training process, it is required that each network find parameters $\boldsymbol{\phi}$ and $\boldsymbol{\theta}$ that will *both minimize* their respective loss functions (This is different from the value function (1) where one seeked to minimize or maximize the same function. By defining separate loss functions, both networks can calculate parameters that minimize the loss.). In particular, in (2), D must find parameters $\boldsymbol{\phi}$ that will make $\mathbf{f}[\mathbf{g}[\mathbf{z_j},\boldsymbol{\theta}],\boldsymbol{\phi}]$ tend to 0. As we are dealing with the negative-log likelihood (i.e., the negative logarithm), if $(1-\mathbf{f}[\bullet])$ evaluates to anything smaller than 1, it will make the first term of $L[\boldsymbol{\phi}]$ positive, and increasingly grow to ∞ as it approaches 1. Hence, it must be lower bounded by 0 to minimize $L[\boldsymbol{\phi}]$. In the second term, $\mathbf{f}[\bullet]$ must tend to 1, as we have seen in the value function (1). As for the generator loss function $L[\boldsymbol{\theta}]$ in (3), the same reasoning of the minimization of the value function applies. In this case, we can see the second term nondependent on $\boldsymbol{\theta}$ is already removed. We can see how the loss functions relate to the value function and how they are applied during the training phase of the neural networks. At every training step, the discriminiator is fed a batch of real images and a batch of generated images on which it is trained on. A visual representation of the process is shown in Fig. 2.

[1] In addition, the sigmoid is added around $\mathbf{f}$ to ensure output are between 0 and 1.

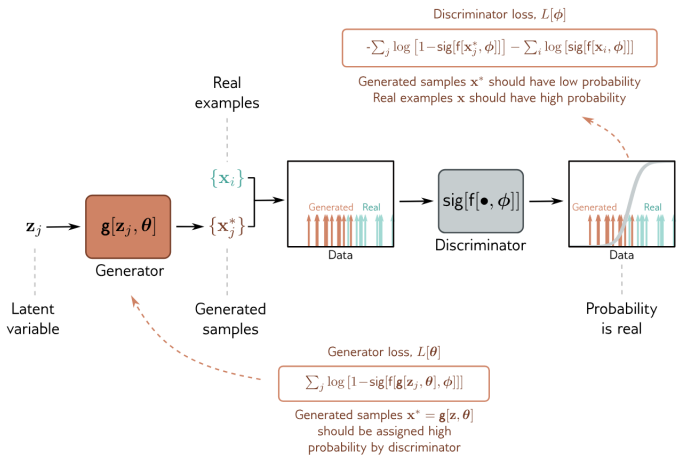


Fig. 2: GAN loss functions. A latent variable $\mathbf{z}_j$ is drawn from the base distribution and passed through the generator to create a sample $\mathbf{x}^*$. A batch $\mathbf{x}_j^*$ of samples and a batch of real examples $\mathbf{x}_i$ are passed to the discriminator, which assigns a probability that each is real. The discriminator parameters ϕ are modified to assign high probability to the real examples and low probability to the generated samples. The generator parameters θ are modified to "fool" the discriminator into assigning the generated samples a high probability. Source: [8]

In this way, the two models compete in this "adversarial" game for some number of iterations. After being trained on the specified number of iterations, the generator model is saved and the discriminator discarded [8]. An extension to the problem is the conditional GAN, forcing the generator to respect certain conditions for generation. In this alternatative training method, the generator and discriminator loss functions are given a new vector of the conditions required for generation and discrimination, such that both the generative and discriminative tasks adhere to the conditions imposed [8].

B. A Review of Existing GAN Models

The clearly defined use case concerning the generation of city traffic maps indicates that relevant papers on the subject could be of use. [7] introduces the Curb-GAN model for traffic prediction, making use of a dynamic convolutional layer that more accurately covers the irregularities of the city roads, as well as a self-attention mechanism, something common to language modelling and machine translation problems. The data used in Curb-GAN however covered the span of 6 months (much less than our available data), and made use of average traffic speed as a condition to the conditional GAN model.

We turn to more generalized GAN models trained on various different kinds of datasets which tend to learn a wider variety of features. For instance, the model in [10] defines the concept of image-to-image translation, where one representation of a scene is converted into another (e.g., day to night, BW to colour, hand sketch into realistic image). One of their relevant examples is the aerial-to-map translation, turning a satellite image into street navigation-like maps, and vice-versa. One notable example taking inspiration from image-to-image translation is [11], where the model generates "synthetic" city maps, employing "water areas mark" for large bodies of water as conditions in the conditional GAN. Lastly, we find FastGAN from [5], which focuses on high-fidelity few-shot image synthesis, to be ideal for our data. Requiring low computing resources, the model proves to be very capable of generating close-to-original images. Of the 12 datasets it is trained on, the model outperforms notable models with 10 of the datasets. Of interest to us is training FastGAN on map data, something which the original paper does not attempt. FastGAN makes two important contributions:

- A Skip-Layer Excitation (SLE) module
- A Self-Supervised Discriminator

The SLE module shares some of the similarities as the ResBlock introduced in ResNets [9]. In simple CNNs, the convolution pass from layer-to-layer is sequential, where the output of one layer is passed directly to the next layer, and that output passed to the following layer, and so on. This can be interpreted as a "series of nested functions" [8]:

$$\mathbf{y} = \mathbf{f}_4[\mathbf{f}_3[\mathbf{f}_2[\mathbf{f}_1[\mathbf{x}, \boldsymbol{\phi}_1], \boldsymbol{\phi}_2], \boldsymbol{\phi}_3], \boldsymbol{\phi}_4] \tag{4}$$

For deep networks, however, consecutive derivates (or gradients) of this network can behave unexpectedly, causing the shattered gradients problem. To this end, ResBlocks have shown to reduce considerably the shattered gradients problem [8]. Residual connections achieve this by adding the input back into the output of the layer. Instead, the authors of [5] apply channel-wise (e.g., Red, Green, Blue, or learnt patterns) multiplication between two distant resolutions (i.e., 8^2 and 128^2) [5]. By design, the SLE module is made to reduce the lower resolution to a spatial dimension of 1×1 while retaining the channel information. This reduction improves training time as less computations are required. This process strengthens the gradient flow similar to ResBlocks but allows the network to achieve similar results while maintaining low computational burden.

In the self-supervised discriminator, the authors in [5] make use of two decoders in the last scales of the discriminator, one at 16^2 and another at 8^2. The decoding process upsamples images from the later scales to 128×128 resolution. Samples at this resolution are evaluated with the auto-encoder loss function in [5], comparing a randomly cropped and a non-croppped image from the real data with a randomly cropped and non cropped image from the two later scales that are passed through the encoder. This auto-encoding forces the discriminator to learn features the encoders can reconstruct.

III. Dataset Description and Preprocessing

The first part of our workflow presented in Fig. 1 requires a dataset of coordinate datapoints for common EV routes.

- We define a datapoint to be a latitude-longitude coordinate pair from our initial dataset.

- We define a route to be a path between a source datapoint and a destination datapoint (i.e., a path from one landmark to another).

Routes are usually paths between two landmarks (e.g., home, work, supermarket). Our initial data only contain landmarks. This is because our starting data are from [3], simulated according to the "motif-based" scheme for EV charging demand. As such, we have no knowledge of the minute-by-minute coordinates during an EV's route.

We give a short description of the initial dataset. The columns include EV battery information and energy efficiency, among other things. We disregard these and only keep columns about departure time, source coordinates, and destination coordinates. As data for arrival time was not collected, we only take departure time into consideration for the time component.

IV. Methodology

A. Route Simulation

The initial dataset is limited in that only coordinate data of landmarks were registered. As a result, there are only a few datapoints registered per EV. It thus becomes necessary to simulate *plausible* routes between these points. A plausible route is a route the driver might have taken from point A to B. As we have no knowledge of this route exactly, we leave it up to the routing API to find a shortest path between the two points. The routes are simulated with the OpenRouteService (ORS) API [12]. To overcome the web API rate limit, the simulations are performed locally, hosted on a local Docker container [13]. To this end, this process effectively augments the number of datapoints between two landmarks, allowing us to plot the routes with more granularity such that the routes resembles connected roads on the map.

B. B-Spline Reparameterization

One caveat of the ORS API is that it returns information much like that of a consumer-grade GPS. This includes text instructions indicating turns, distance, alerts, among other things. Ignoring this information is trivial, but the problem arises in that the many individual coordinates along the simulated route are placed according to the location of the relevant navigation indications. In essence, the end result is that portions of the route with more turns and instructions are more heavily populated with coordinate points than less instructions-filled portions. As such, long stretches of highway driving may have few coordinate points in the region despite being very well-travelled by motorists. Similarly, rural portions of the route with many turns and navigation instructions will be densely populated with coordinate points despite being infrequently traveled. This poses a problem during the map shading process which relies on the density of localized points to represent traffic density. It becomes necessary to ensure all routes are equally dense in points such that only overlaping routes highlight greater traffic density. Our motivation to recalculate equidistant coordinate points arises from this issue.

$$F(t) = \sum_i c_i B_{i,k}(t) \tag{5}$$

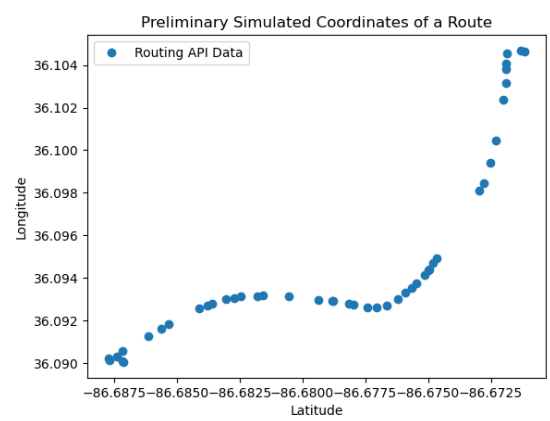
A basis spline (or B-spline) (5) is a piecewise polynomial of order k defined by its coefficients and knots, where $k-1$ gives the degree of the polynomial (itself a B-spline) multiplied by some constant c. Note that each knot t is a parameter *along* the curve, at which the B-spline is evaluated. The piecewise nature of the B-spline allows it to fit the nonfunction-like shape of the routes. Using the de Bloor algorithm [14], we interpolate an order 4 (degree 3) B-spline from the list of coordinate points returned for a simulated route, as shown in Fig. 3b.

$$B_{i,1}(t) = \begin{cases} 1 & \text{if } t_i \leq t < t_{i+1} \\ 0 & \text{otherwise} \end{cases} \tag{6}$$

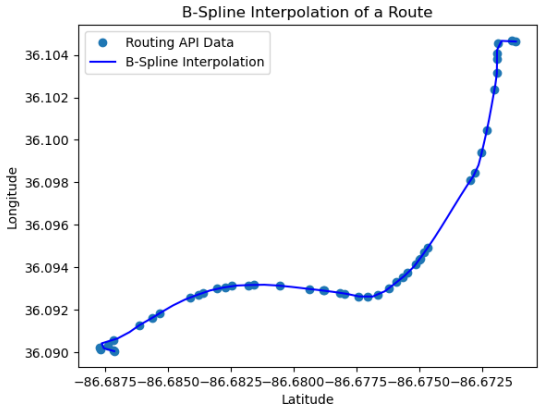
$$B_{i,k}(t) = \frac{t - t_i}{t_{i+k} - t_i} \cdot B_{i,k-1}(t) + \frac{t_{i+k} - t}{t_{i+k} - t_i} \cdot B_{i+1,k-1}(t) \tag{7}$$

The de Bloor algorithm from [14] is shown above. In the base case where $k = 1$, we have (6). This is a B-spline of order of k, meaning it is of degree 0. Anything to the power of 0 will give 1, as shown. For $k > 1$, we have (7), calculated recursively.

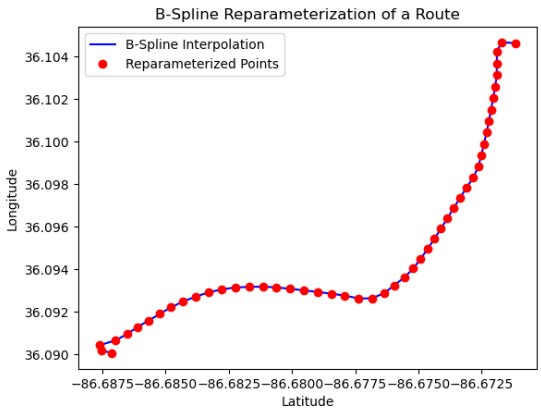
In this final stage, we evaluate the interpolated B-spline over a new set of parameters which we call u sampled over the interval $[0, 1]$. (This interval is fixed and represents the "progression" along the curve. It is the number of parameters and their location that we alter). From trial and error, we find 100 parameters to give acceptable results for most routes, as seen in Fig. 3c. A lesser number of parameters will give a more sparse curve with less densely packed points, whereas a greater number of parameters a denser curve. In this way, we are able to evaluate the B-spline at u, defining new parameters along the curve, and thus reparameterizing it as needed. When using the Scipy Python package [15], we are returned with a set of proper x and y pairs (longitude and latitude for our purposes) which is of the same length as u, as expected. These data are from which our maps are shaded. To summarize, we first calculate a B-spline interpolation of the given path, we



(a)



(b)



(c)

Fig. 3: The three step B-spline reparameterization process with the simulated route data from ORS (a), followed by the B-spline interpolation (b), and the final reparameterization (c). 50 parameters are used in this figure for clarity.

specify a new set of parameters u, and we evaluate the B-spline at u to obtain more equidistant coordinate points along the route. This process could be improved as some curves do not quite produce as equidistant points as we would like but offers an overall good improvement over the baseline given by the ORS API.

C. Map Shading

As mentioned in the previous section, the map creation works through a shading process. It paints the map by shading an area around closer-proximity points more intensely than areas with more sparse points. The map "canvas" is divided into bins, and bins having more points inside are shaded brighter than bins having less points landing inside of them. Note that a single road could be used in multiple routes by different EVs. A road that is used with higher frequency will have multiple overlapping routes, and thus a higher count of points (from each of those routes). With this in mind, said road will be shaded brighter, effectively capturing overlapping routes of individual EVs. As we reduce the time interval between maps, we more accurately capture the real traffic density at a given time. Through this method, we attempt to approximate traffic density of the EVs' travels. For every hour of data in a day, a 256×256 map is produced, for 30 days worth of data. This totals 588 map images (for many days < 24 hours of data was collected).

D. GANs and FastGAN Training

We adopt the FastGAN model to train on our map dataset. In particular, we choose it in order to make use of its good performance on few-shot image synthesis. The model was trained as follows:

- 50k iterations
- 588-sample dataset
- Batch size of 8
- 1 hour on an H100 GPU

We record the model's synthesis performance with the Fréchet Inception Distance (FID) [16]. This is inline with the original paper [5].

V. Simulation Results

Images were generated at 256×256 for each iteration the model was trained on. Checkpoints were saved at every 10k iterations, for a total of 5 checkpoints. The FID is reported for each iteration and compared in Table I. Qualitave results are shown in Fig. 4 and 5. Note that while the training takes 1h for 50k iterations, the model performance peaks at 30k, meaning that it could perform adequate results in even less time. Although the dataset includes images captured for different times of day, the variability of traffic density in the images is still low from one hour to the next. This was shown in the generated images as none of them included maps with few routes in them. That is to say, the model captures the more prominent features of the dataset. Despite this, we can see the model's attempt to introduce different transformations (albeit simple) during generation by examining Fig. 4. Many of the generated samples shown are simple rotations, mirroring, or a combination of both. This leads us to hypothesize the model could learn from additional map datasets of similar size as ours to gain more exposure to different city shapes and traffic patterns along different kinds of roads.

TABLE I: Training Performance Metrics by FID

Iteration Checkpoint	10 000	20 000	**30 000**	40 000	50 000
FID	479.12	532.87	**152.43**	167.35	253.15

VI. Conclusion

We present a workflow leveraging GANs to build large map-like city traffic images from a small amount of coordinates of frequently-visited landmarks by EVs. We expand the initial basic coordinate dataset by calculating plausible routes with a routing API. We correct for a coordinate distribution imbalance along the route by reparameterizing the curve over a specified interval. From these data, a shaded map dataset is

created. The resulting dataset is used to train FastGAN and generate images of similar types, showing a decent ability of generalizing and expanding the original dataset. We hope these methods and data can be used in future research where EV travel data are limited or difficult to obtain.

REFERENCES

[1] M. M. Taamneh and H. Y. Makahleh, "The prospects of adopting electric vehicles in urban contexts: A systematic review of literature," Transportation Research Interdisciplinary Perspectives, vol. 31, 101420. DOI:10.1016/j.trip.2025.101420

[2] A. Pamidimukkala, S. Kermanshachi, J. M. Rosenberger, and G. Hladik, "Adoption of electric vehicles: A state-of-art review." Transportation Research Procedia, vol. 73, pp. 313-320, 2023, DOI:10.1016/j.trpro.2023.11.923

[3] M. Rashid, T. Elfouly, and N. Chen, "Travel motif-based learning scheme for electric vehicle charging demand forecasting," 2023 IEEE Vehicle Power and Propulsion Conf. (VPPC), Milan, Italy, 2023, pp. 1-6, DOI:10.1109/VPPC60535.2023.10403362

[4] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, Y. Bengio, "Generative Adversarial Networks," 2014, arXiv:1406.2661. [Online]. Available: https://arxiv.org/abs/1406.2661

[5] B. Liu, Y. Zhu, K. Song, and A. Elgammal, "Towards faster and stabilized GAN training for high-fidelity few-shot image synthesis," 2021 Int. Conf. on Learning Representation, Poster, Jan. 2021, [Online] Available: https://arxiv.org/pdf/2101.04775

[6] T. Karras, S. Laine, M. Aittala, J. Hellsten, J. Lehtinen and T. Aila, "Analyzing and improving the image quality of StyleGAN," 2020 IEEE/CVF Conf. on Comp. Vision and Pattern Recognition (CVPR), Seattle, WA, USA, 2020, pp. 8107-8116, DOI:10.1109/CVPR42600.2020.00813

[7] Y. Zhang, Y. Li, X. Zhou, X. Kong, and J. Luo. 2020. "Curb-GAN: Conditional urban traffic estimation through spatio-temporal generative adversarial networks." In Proc. of the 26th ACM SIGKDD Int. Conf. on Knowledge Discovery & Data Mining (KDD '20). Association for Computing Machinery, New York, NY, USA, pp. 842-852. DOI:10.1145/3394486.3403127

[8] Prince, Simon J. D. Understanding Deep Learning. The MIT Press, 2023.

[9] K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 2016, pp. 770-778, doi: 10.1109/CVPR.2016.90.

[10] P. Isola, J. -Y. Zhu, T. Zhou and A. A. Efros,"Image-to-Image Translation with Conditional Adversarial Networks," 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Honolulu, HI, USA, 2017, pp. 5967-5976, DOI: 10.1109/CVPR.2017.632

[11] A. Albert, E. Strano, J. Kaur, and M. C. Gonzalez, "Spatial sensitivity analysis for urban land use prediction with physics-constrained conditional generative adversarial networks," 2019, arXiv:1907.09543. [Online] Available: https://arxiv.org/abs/1907.09543

[12] Heidelberg Institute for Geoinformation Technology, "Open Route Service," https://openrouteservice.org/ (Accessed Dec. 16, 2025)

[13] Open Route Service, "Running with Docker," https://giscience.github.io/openrouteservice/run-instance/running-with-docker (Accessed Dec. 16, 2025)

[14] C. de Boor, "On calculating with B-splines," Journal of Approximation Theory, vol. 6, no. 1, pp. 50-62, July 1972. DOI:10.1016/0021-9045(72)90080-9

[15] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, CJ Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E.A. Quintero, Charles R Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. (2020) SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. Nature Methods, 17(3), 261-272. DOI: 10.1038/s41592-019-0686-2.

[16] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter. "GANs trained by a two time-scale update rule converge to a local nash equilibrium." In Proc. of the 31st Int. Conf. on Neural Information Processing Systems (NIPS'17). Curran Associates Inc., Red Hook, NY, USA, pp. 6629-6640. DOI:10.5555/3295222.3295408

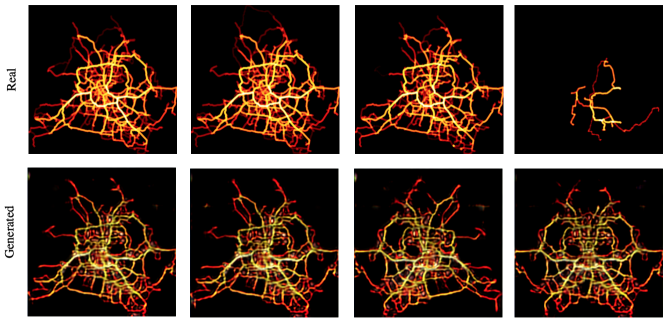


Fig. 4: Comparison of real (top row) and generated images (bottom row). The right-most image of the top row is one of few real images with few routes. Since they were limited in numbers, the model did not seem to capture them very well.

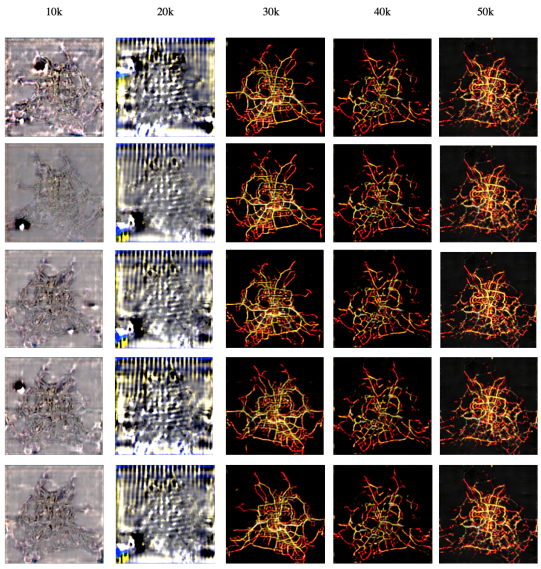


Fig. 5: Comparison of images generated at each saved iteration checkpoint. At 30k iterations, the model performed the best.